VIRS original system: inventory and reasons for change

Prepared September 15, 2026, for the product owner and faculty advisor. This document describes **the original Vocabulary in Reading Study (VIRS) application** as it existed before any migration work: how the project was organized, why it was built that way, what works today, what does not and why, everything it depends on, what it costs to run, and why it needs to change. The last section compares it with the proposed setup.

Acceptance checklist

- [x] A written inventory exists listing:
  - [x] what currently runs without modification (section 5)
  - [x] what's broken and why (section 6)
  - [x] all dependencies: languages, frameworks, libraries, versions (section 7)
- [x] The reasons the system needs to change are documented (section 8)
- [x] The cost of the original setup, and why the proposed update is better (sections 9–10)

How this was verified. On September 18, 2026 the original application was **built and run end to end**, and every feature was exercised in a real browser. Nothing in the application code was changed for the audit; the frontend was restored to its original pre-migration state first.

The environment used:

Frontend	`apps/web` (original `VIR-Frontend`), Angular dev server on port 4200, Node 8.17.0
Backend	`apps/api` (original `VIR-Backend`), the packaged 6.3.0 jar on port 8080, JDK 8 (Corretto 1.8.0_504)
Processor	Apple Silicon, running the backend under **Rosetta x86 emulation** (required: see 6.1)
Database	MySQL 26.7 on port 3307, loaded with the project's own `data.sql` and `updateGrades.sql` (2,660 words, 1,867 with grades)
Not configured	AWS credentials, Tesseract, SendGrid — the same gaps a new deployment would have

A team member had already reported that everything worked except image upload. **This audit agrees with that**, and adds the cause of the image failure plus a handful of quieter defects that are easy to miss in normal use. Items below are labelled **observed** (seen in the running application) or **code-level** (found by reading the source and not reproducible in this environment).

---

1. What VIRS does

VIRS helps teachers, students and researchers judge how difficult a text is by looking at its vocabulary. The original application lets users:

- **Analyze a text:** typed, a Word document, a PDF, or a photo of a page.
- **See the enhanced text:** every word colored by the research word list it belongs to (K1, K2, K3, Academic, Basic Academic, STEM, high, medium and low frequency, or off-list), with a definition on click.
- **See statistics:** Flesch reading-ease score, reading level, and word counts and percentages per list, with charts.
- **Browse word lists:** search by list and grade (Kindergarten–12), and download lists as CSV/Excel.
- **Translate** text (iTranslate).
- **Manage an account:** register, log in, edit the profile, change or recover the password, delete the account.
- **Administer the word database:** add, edit, delete, bulk CSV upload, delete a whole list.
- **Other pages:** contact form, references, "Cite us", donate link, and a language switch for navigation labels (English, Spanish, Farsi).
- **Vocabulary tests:** built but hidden. The sidebar comments them out ("TEMPORARILY HID VOCABULARY TESTS AT P.O's REQUEST"), and the test banks are marked "THIS SECTION IS NOT DONE YET".

---

2. How the original project was divided

VIRS was built as **two separate projects** in one parent folder (`virs/Code/` , per the project READMEs), plus a database and several external services.

Part	Original project	Technology	Role
Frontend	`VIR-Frontend` (now `apps/web`)	Angular 4 single-page app, TypeScript 2.3, Bootstrap 4, SCSS	All screens; calls the backend's `/api/*` endpoints
Backend	`VIR-Backend` (now `apps/api`)	Spring Boot 1.5 REST API, Java 8, Maven	Text analysis, file parsing and OCR, dictionary lookups, word lists, user accounts, admin
Database (production)	AWS	MySQL (credentials encrypted with Jasypt in `application.yml`)	Word lists (about 42,000 rows per the schema's `AUTO_INCREMENT`), users, password-reset tokens

Part	Original project	Technology	Role
Database (development)	Each developer's machine	Local Postgres (`localhost:5432/vir`), schema created by Hibernate (`ddl-auto: update`)	Local testing
Word-list data	`VIR-Backend/src/main/resources/*.sql`	MySQL SQL scripts (`schema.sql`, `data.sql`, `queriesEWL.sql`, `updateGrades.sql`)	Loaded into the database by hand

How the pieces connected

During development, the two projects ran side by side:

```
Browser → Angular dev server (localhost:4200) —proxy /—▶ Spring Boot (localhost:8080) —▶ local database
```

`VIR-Frontend/proxy.conf.json` forwards every request to port 8080, and a `dev`-only CORS configuration in the backend allows the cross-origin calls.

In production, they shipped as one unit:

```
Browser → AWS Elastic Beanstalk (nginx) → Spring Boot jar ─┬─ /api/*   → REST controllers → MySQL
                                                           └─ other URLs → Angular build (inside the jar)
```

1. The Angular app was built (`ng build --prod`) and its output copied into `VIR-Backend/src/main/resources/static/`.
2. Maven packaged the API and the frontend into a single jar. The `prod` profile excluded the SQL scripts and assembled an Elastic Beanstalk bundle with a generated `Procfile`.
3. `AngularRedirectController` forwarded any non-file URL to `index.html`, so Angular's routes (e.g. `/search-words`) worked on refresh.
4. An `.ebextensions` nginx setting raised the upload limit above 1 MB so documents and images could be analyzed.

---

3. Why it was built this way

The code and configuration show the reasoning behind most choices:

Decision	Why it made sense at the time
Angular 4 + Spring Boot 1.5	A mainstream, well-documented full-stack pairing in 2017 (the site footer reads "Copyright © 2017"). The frontend started from an Angular/Bootstrap admin seed template.
Two projects, one deployable jar	Frontend and backend developers could work independently, while production needed only one server and one deploy. With both on the same origin, no CORS setup was needed in production.
AWS Elastic Beanstalk	Runs a Java jar with little server administration, and sits next to AWS databases and AWS Textract.
MySQL	The common default for Java web apps on AWS; the SQL scripts and entity definitions are written for it.
Jasypt-encrypted properties	Kept production passwords and API keys out of plain text while still committing `application.yml` to the repository.
Hibernate `ddl-auto: update` (development)	Tables are created automatically from the Java classes, so developers don't maintain SQL migrations.
Apache Tika	One library reads text from PDFs and Word documents.
Tesseract + OpenCV, later AWS Textract	Tesseract and OpenCV give free, self-hosted OCR with image clean-up. Textract was added later for better accuracy on photos, with a 1,000-per-month counter to control cost.
Oxford Dictionaries + Wiktionary	Two definition sources: a curated commercial one and a free one.
SendGrid (plus Gmail SMTP configuration)	Transactional email for password resets and emailed error reports without running a mail server.
Client-side AES password encryption	Appears intended to avoid sending plain-text passwords. Because the key ships inside the browser code, it provides no real protection (section 6.3).
formcarry.com, Google Translate (browser), PayPal	Contact form, translation and donations with no backend code.
Spring profiles `dev` / `test` / `integration-test` / `prod`	Different databases, logging and packaging per environment.

---

4. External services the original app depends on

Service	Used for	Current status
AWS Elastic Beanstalk	Hosting the jar	Paid, always-on server
MySQL on AWS	Production data	Credentials only readable with the Jasypt vault key
AWS Textract (Java SDK v1)	Text from images	**Not configured — this is why image upload fails** (6.2 #1); SDK v1 is end-of-support

Service	Used for	Current status
Tesseract OCR (native install)	Text inside PDFs	Must be installed on the server (`TESSERACT_PATH`, `TESSDATA_PREFIX`)
OpenCV 3.2 (bundled native library)	Image clean-up before OCR	No ARM support (verified)
Oxford Dictionaries API v1	Definitions	Discontinued by Oxford
Wiktionary API	Definitions (the source the UI actually uses)	**Now rejects the app's requests with HTTP 403** (6.2 #3)
SendGrid	Password-reset emails and error reports	API key lost since 2021 (per the backend README)
Gmail SMTP	Configured in `application.yml` (`SendEmail` service)	Never called by the app; its password was committed to the repository
Google Translate (unofficial endpoint)	iTranslate and the image flow, called from the browser	Unofficial, rate-limited (HTTP 429 observed)
formcarry.com	Contact form	Third-party form service
Adobe Typekit, PayPal	Header font, donate link	External links

---

5. What works

The application runs and the great majority of it works. Exercised in the browser against the running backend:

Feature	Result
Dashboard	✅ Loads
Analyze typed text → **Enhanced Text**	✅ 106 words colour-coded by list in the sample text
Analyze typed text → **Statistics**	✅ Readability score, level ("Upper intermediate"), per-list table and both charts
Word document upload (.docx)	✅ 16 words extracted and analyzed
PDF upload	✅ 7 words extracted and analyzed
Image upload	❌ "The file is not readable" (6.2 #1)
Search word lists	✅ Lists load; prefix search for "ab" returned *about, above*
Download an individual list (K1)	✅ 999 words
Download a group of lists (K1–K3)	✅ 1,999 words, both populated lists included
Register an account	✅

Feature	Result
Log in	✅
My Profile	✅
Change password	✅ (`PATCH` → 200)
Delete account	✅ Server returns 204 and the UI confirms deletion
Admin panel (after the Spring Security login)	✅ Search works; add/edit/delete work, with the caveat in 6.2 #2
iTranslate	✅ "Good morning, how are you?" → "Buenos días, ¿cómo estás?"
Contact and References pages	✅
Definition pop-up	❌ "We couldn't find the definition" (6.2 #3)
Results after a page refresh	❌ "Opps! Please go back" (6.2 #7)

Build and run checks:

Component	Command	Result
Frontend production build, Node 8	`ng build --prod`	✅ Builds in about 35 s
Frontend dev server, Node 8	`ng serve`	✅ Compiles and serves, proxying `/api` to port 8080
Frontend build, modern Node (26)	`ng build --prod`	❌ `error:0308010C: digital envelope routines::unsupported` — webpack 3's hashing is rejected by OpenSSL 3, used by every Node release from 17 onward
Backend compile and package, JDK 8	`./mvnw package -DskipTests`	✅ 91 source files → a 285 MB jar with 296 bundled libraries
Backend start-up (x86, MySQL, prerequisites met)	`java -jar VIR-Backend-6.3.0.jar`	✅ Starts in 34 s, serves on port 8080
Backend tests	`./mvnw test`	⚪ Skipped by default (`maven.test.skip=true` in the `dev` profile)
Frontend tests	Karma / Protractor	⚪ Not run; mostly generated boilerplate specs

In short: the software works. What follows is not an argument that it is unusable, but a record of the conditions it now demands (6.1), the defects that remain (6.2), and the security problems that make it unsafe to expose publicly (6.3).

---

6. What's broken, and why

6.1 What it takes to run it (observed)

Getting the application running took most of a working session. Each item below is a hard prerequisite — the app stops at each one in turn until it is satisfied:

1. **Tesseract OCR must be installed locally.** Without `TESSDATA_PREFIX` and `TESSERACT_PATH`, start-up aborts with `Could not resolve placeholder 'TESSDATA_PREFIX'` — even for features that never touch OCR.
2. **The Jasypt vault key must be supplied.** `DecryptionException: Decryption of Properties failed`. The Oxford and production database credentials are encrypted inside `application.yml`, so `APP_ENCRYPTION_PASSWORD` has to come from outside the repository. For this audit the encrypted values were bypassed with dummy environment variables. **If that key is lost, the production credentials cannot be recovered.**
3. **The processor must be x86.** `UnsupportedOperationException: Architecture "aarch64" is not supported` — OpenCV 3.2.0 (2016) ships no ARM binaries. On Apple Silicon the backend only started under **Rosetta emulation with an x86 JDK**. Any ARM machine or ARM cloud instance cannot run it natively.
4. **The database must be MySQL.** The committed configuration points the datasource at Postgres, but on Postgres Hibernate's generated `create table user (… creation_date DATETIME … locked BIT DEFAULT 0 …)` is rejected with `syntax error at or near "user"`: `user` is reserved there and those column types are MySQL-only. Accounts therefore work only on MySQL, and the `data.sql` / `updateGrades.sql` seed scripts are MySQL-only as well.
5. **A modern MySQL needs the connection relaxing.** MySQL 26.7 no longer offers the authentication plugin the 2018 connector expects, and the 2018 connector rejects the server's certificate. The connection only succeeded with `useSSL=true&verifyServerCertificate=false&allowPublicKeyRetrieval=true` — a weakened TLS setting.

None of this is a defect in the application's logic. It is the cost of keeping it alive: a specific processor, a specific database, a native OCR install, a secret key, and two runtimes that no longer receive security updates.

6.2 Defects

Ordered by how much they matter. "Observed" means reproduced in the running application during this audit.

#	Problem	Evidence	Cause
1	**Image upload fails.** The UI shows "Oh snap! The file is not readable."	**Observed.** Backend log: `com.amazonaws.SdkClientException: Unable to find a region via the region provider chain.`	Image analysis calls **AWS Textract**, which needs an AWS region and credentials in the server's environment. None are configured. The failure is swallowed and an empty result returned, so the UI reports an unreadable file rather than a configuration problem. This is almost certainly the same failure the team saw. **It is a configuration issue, not broken code** — but it also means image analysis depends on a billable AWS account being wired up correctly.
2	**Adding a word to a second list silently removes it from the first.**	**Observed** end to end: "water" was in K1; adding it to STEM in the admin panel showed "water is added to stem category", after which the database held **only** `water → stem`. K1 quietly lost a word.	The admin save looks a word up by its value alone and overwrites that row instead of inserting a new one. Repeated use erodes the word lists, and the UI reports success each time.
3	**Dictionary definitions never load.** Every lookup shows "We couldn't find the definition, please try another word."	**Observed:** `GET /api/entries/book?source=WIKI` → 503. The underlying cause, reproduced from the app's own Java runtime: Wiktionary replies **HTTP 403**.	The app sends no User-Agent header, and Wikimedia's API rejects such requests. A separate, older problem sits behind it: Wiktionary's markup has since changed from `<h2><span id="English">` to `<h2 data-mw-anchor="English">`, which the cleaner does not recognise, so even a successful response would be returned unfiltered — every language section included (the live page for "book" now carries Beja, Dutch and others).
4	**Words followed by punctuation are not recognised, and hyphenated entries never match.**	**Observed:** analyzing `books. book books co-worker coworker` returned *books.* → off-list, *book* → K1, *books* → K1, *co-worker* → off-list, *coworker* → off-list — although the word list itself contains hyphenated entries such as `super-power` and `self-review`.	Singularization runs before punctuation is stripped, and hyphens are removed from the text but kept in the stored list. Any word ending a sentence is therefore counted as off-list, which inflates the "Names & off-list" figure in every analysis.
5	**Basic Academic words are dropped from the statistics.**	**Observed:** a word whose only list is `baw` is shown correctly in the coloured text, yet four analyzed words produced a reported total of three.	The counting code has buckets for every list except Basic Academic, and the total is the sum of those buckets. With the current data the effect is hidden — all 200 `baw` words also belong to another list, and the conflict rule always prefers the other one — but any Basic-Academic-only word vanishes from the totals and percentages.

#	Problem	Evidence	Cause
6	**Word lists are downloaded onto the server's disk.**	**Observed:** after the audit, `K1.csv`, `K1–K2–K3.csv` and `awl–baw.csv` were sitting in the backend's working directory.	Each download writes a CSV named after the requested lists into the server's working folder. Two users downloading the same list at once write to the same file.
7	**Refreshing the results page loses the analysis** ("Opps! Please go back").	**Observed.**	Results are held only in memory in the browser, so any refresh, shared link or back-navigation discards them and the text must be analyzed again.
8	**"Download List" on the search page exports only what is on screen.**	**Observed:** with 2 rows displayed after a search, the downloaded `WordList.xlsx` contained those 2 rows.	The export is generated from the rendered HTML table rather than from the database.
9	Password-reset emails and error reports are never delivered.	**Code-level** (not testable here: no SendGrid key).	Both go through SendGrid, whose API key has been lost since 2021 (per the backend README), and SendGrid is forced into sandbox mode unless `PROD=1`. The Gmail sender configured in `application.yml` is never actually called.
10	Oxford dictionary lookups cannot work.	**Code-level.**	The Oxford Dictionaries API v1 has been discontinued. Only the Wiktionary source is reachable from the UI in any case.
11	A readability score that lands between two bands is labelled "Beginner".	**Code-level.**	The level bands cover whole numbers only, so scores in the gaps 29–30, 59–60, 69–70 and 79–80 fall through to the default. Roughly 4% of the range is affected.
12	Two consecutive spaces start a new paragraph in the enhanced text.	**Observed:** double spaces in submitted text produced empty tokens, which the UI renders as line breaks.	Every whitespace character is treated as a token boundary, and empty tokens are drawn as breaks.
13	The Textract monthly spend limit and the iTranslate character limit do nothing.	**Code-level.**	The Textract counter lives in memory, so it resets on every restart and is per-server; the iTranslate "characters left" counter always resets to 1,000.

#	Problem	Evidence	Cause
14	Translation depends on an unofficial endpoint.	**Observed working** during the audit ("Buenos días, ¿cómo estás?"), but an **HTTP 429 rate-limit response** was recorded from this machine on an earlier attempt.	The browser calls an undocumented Google endpoint directly, with no agreement or quota.

Two problems previously suspected from reading the code were **tested and found not to exist**:

- **Group downloads work.** `/api/download/K1, K2, K3` returned 1,999 words spanning K1 and K2 (K3 is absent simply because the dataset contains no K3 words). Spring trims the spaces, so the suspected mismatch never happens.
- **Deleting an account works cleanly.** The server returns 204 and the UI confirms the deletion; the empty response is handled correctly.

6.3 Security problems

The first three were **confirmed against the running application** during this audit, using nothing but a command line and no credentials of any kind.

#	Problem	Evidence
1	**Any visitor can read any user's password.**	`GET /api/user/audituser` with no authentication returned the full record, including `"password":"U2FsdGVkX1+…"`. That value is AES-encrypted with a key hardcoded in the JavaScript bundle (`7061737323313233`), so it can be decrypted by anyone who opens the site. Login works by downloading this value and comparing it in the browser.
2	**The entire user list is public.**	`GET /api/user` with no authentication returned every user with their passwords and email addresses. Only `/api/admin/**` is protected.
3	**Anyone can modify anyone's account.**	An unauthenticated `PATCH /api/user/audituser` changed the account's name to "HACKED BY ANYONE" and its email to an attacker-controlled address; the change was accepted with HTTP 200 and persisted.
4	Passwords travel inside URLs.	`DELETE /api/user/{user}/{password}` and `POST /recovery_password?token=…&password=…` place credentials in paths and query strings, where they are recorded in server and proxy logs.
5	Password-reset endpoints reveal which accounts exist, and reset tokens never expire.	Recovery returns 404 for an unknown user, and no expiry is ever applied to an issued token.
6	Admin is a single shared username and password held in Spring configuration, and the Gmail SMTP password was committed to the repository in plain text.	Access cannot be granted or revoked per person, and the mail account must be rotated.

#	Problem	Evidence
7	Conflicting and outdated libraries.	Two versions of opencsv (2.0 and 3.3) are bundled together; `spring-context-support` 3.1.1 is mixed with Spring 4.3.10; Jackson 2.8.9 and Tomcat 8.5.16 are years behind published security fixes.

Items 1–3 together mean that **anyone who can reach the site can take over every account on it**. This is the strongest single reason not to return the application to the public internet in its current form.

---

7. Dependency inventory

7.1 Languages, runtimes and tools

Item	Version	Notes
Java	8 (`java.version` 1.8)	Required by Spring Boot 1.5
Maven	3.9.9 (wrapper 3.3.2)	Via `./mvnw`
TypeScript	2.3.4	Frontend
Node.js	8.x (verified 8.17.0)	Build needs Node ≤16 (OpenSSL); dev server needs Node <12 (`process.binding`); Node 8 is end-of-life (Dec 2019)
Sass	`node-sass` (original)	No longer installs on current Node/arm64; Dart Sass 1.32.13 was substituted to run it today
SQL	MySQL dialect (production scripts), Postgres (local development)	

7.2 Backend (`VIR-Backend`): Spring Boot 1.5.6, resolved versions from the built jar

Area	Library	Version
Framework	spring-boot (starter-parent)	1.5.6.RELEASE (end-of-life Aug 2019)
	spring-core / spring-web / spring-orm	4.3.10.RELEASE
	spring-context-support	3.1.1.RELEASE (conflicts with Spring 4.3)
	spring-security-core	4.2.3.RELEASE
	spring-data-jpa	1.11.6.RELEASE
	spring-boot-starter-mail / javax.mail	1.5.6 / 1.5.6
Web server	tomcat-embed-core	8.5.16 (Tomcat 8.5 end-of-life Mar 2024)
Persistence	hibernate-core	5.4.18.Final (forced; Boot 1.5 expects 5.0)
	hibernate-validator	5.3.5.Final
	HikariCP	2.5.1

Area	Library	Version
	mysql-connector-java	5.1.47
	postgresql	42.2.27
	sqlite-jdbc	3.15.1
JSON / config	jackson-databind	2.8.9
	snakeyaml	1.17
	jasypt / jasypt-spring-boot-starter	1.9.2 / 1.16
API docs	springfox-swagger2 / swagger-ui	2.7.0
Text & files	tika-core / tika-parsers	1.16 (Tika 1.x end-of-life)
	pdfbox	2.0.6
	opencsv	3.3 **and** 2.0 (both bundled)
	commons-csv	1.0
	commons-lang3	3.4
	jsoup	1.11.3
	thucydides-core (for its word inflector)	0.9.275
	guava	26.0-jre
Images / OCR	aws-java-sdk-textract / core	1.11.637 (AWS SDK for Java v1, end-of-support)
	tesseract-platform (JavaCPP)	4.0.0-rc2-1.4.3
	leptonica	1.76.0-1.4.3
	javacpp	1.4.3
	opencv (openpnp)	3.2.0-1 (no ARM binaries)
	imgscalr-lib	4.2
	jai-imageio-core / jpeg2000	1.3.1 / 1.3.0
	levigo-jbig2-imageio	2.0
Email	sendgrid-java / java-http-client	4.3.0 / 4.2.0
Logging	logback-classic	1.1.11
Testing	spring-boot-starter-test	1.5.6 (managed)

In total the jar bundles **296 libraries (285 MB)**, including native binaries for platforms the app never ran on.

7.3 Frontend (`VIR-Frontend`): resolved versions

Area	Package	Version
Framework	@angular/core, common, compiler, forms, http, router, animations, platform-browser(-dynamic)	4.4.7
	@angular/cli	1.7.4 (webpack 3)
	@angular/compiler-cli, language-service	4.4.7
	rxjs	5.5.12
	zone.js	0.8.29
	core-js	2.6.12
UI	@ng-bootstrap/ng-bootstrap	1.0.0-beta.6
	Bootstrap (via the seed template's SCSS)	4.0.0-alpha.6 (per the frontend README)
	font-awesome	4.7.0
	popper.js	1.16.1
	chart.js / ng2-charts	2.9.4 / 1.6.0
	ngx-infinite-scroll	0.8.4
	ng2-simple-timer	1.3.5
i18n	@ngx-translate/core / http-loader	7.2.2 / 0.1.0
Utilities	crypto-js / @types/crypto-js	3.3.0 / 3.1.47
	crypto-browserify, buffer	3.12.1, 5.7.1
	xlsx	0.16.9 (abandoned on npm; known vulnerabilities)
	file-saver	2.0.5
	google-translate-api	2.3.0
	angular-1.6-cli	1.6.3 (unused)
	mocha	5.2.0 (unused at runtime)
Styles	node-sass	original dependency (see 7.1)

Area	Package	Version
Tests / lint	karma 1.7.1, karma-chrome-launcher 2.1.1, karma-cli 1.0.1, karma-jasmine 1.1.2, karma-jasmine-html-reporter 0.2.2, karma-coverage-istanbul-reporter 1.4.3, karma-html-reporter 0.2.7, karma-junit-reporter 1.2.0, jasmine-core 2.6.4, jasmine-spec-reporter 4.1.1, @types/jasmine 2.5.45, protractor 5.1.2, ts-node 3.0.6, tslint 5.3.2, codelyzer 3.0.1, uglify-js 2.8.29, @types/node 6.0.118	
Language	typescript	2.3.4

52 direct packages in total.

---

8. Why the system needs to change

The application works, and this section does not claim otherwise. The case for change rests on four things the audit made concrete.

1. **Anyone who can reach the site can take over every account on it.** Passwords are readable without logging in, the whole user list is public, and any account can be edited by an anonymous request — all three confirmed on the running application (6.3). No amount of dependency updating fixes this; the login design itself has to change.
2. **Keeping it running is getting harder every year.** It needs x86 hardware (it will not start on Apple Silicon or ARM servers without emulation), MySQL specifically, a local Tesseract install, a secret key stored outside the repository, and two runtimes — Node 8 and Java 8 — that stopped receiving security updates in 2019. Connecting it to a current MySQL required deliberately weakening the TLS settings (6.1). Every new developer and every server rebuild pays this cost.
3. **Parts of it depend on services that have moved on without it.** Image upload fails because AWS Textract is not configured; dictionary lookups fail because Wikimedia now rejects the app's requests outright; password-reset email cannot be sent because the SendGrid key has been lost; the Oxford dictionary API no longer exists (6.2). These are not one-line fixes — each is an integration that has to be rebuilt.
4. **Quiet data damage is happening in the admin panel.** Adding a word to a second list removes it from the first, while reporting success (6.2 #2). The word lists are the project's core research asset, and they are being eroded by ordinary use.

Alongside those: the readability and word-matching defects skew the numbers the tool exists to produce (6.2 #4, #5, #11), the hosting is a paid always-on server with manual deploys (section 9), and the stack — Spring Boot 1.5, Angular 4, Tomcat 8.5, Tika 1.x, AWS SDK v1 — is uniformly out of support, which makes both security patching and hiring harder.

---

9. Cost of the original setup

The repository doesn't include the AWS account's billing or the exact server sizes (the Elastic Beanstalk environment settings were never committed). The figures below are therefore **estimates**, based on what the application needs to run and on approximate AWS on-demand list prices (us-east-1). Confirm them against the actual AWS bills.

What the app needs: a Java 8 server that loads a 285 MB jar with Tesseract and OpenCV native libraries into memory. Realistically that means at least 2 GB of RAM on an x86 machine, running 24/7, plus a managed MySQL database.

Item	Typical choice for this app	Approx. monthly cost
Elastic Beanstalk (the service itself)	—	$0 (you pay for the resources below)
Application server (EC2)	t3.small (2 GB) to t3.medium (4 GB), always on	$15 – $30
Load balancer (if the environment is load-balanced)	Application Load Balancer	$0 – $20
Database (RDS MySQL)	db.t3.micro to db.t3.small, 20 GB storage	$15 – $28
Disk, backups and public IP address	EBS volume, snapshots, IPv4 address	$5 – $10
Data transfer out	Low traffic	$0 – $5
Estimated total for hosting		**about $35 – $95 per month ($420 – $1,140 per year)**

Usage-based and other costs on top of hosting:

Item	Cost
AWS Textract (image analysis)	About $1.50 per 1,000 images. The app's monthly limit doesn't work (6.2 #15), so there is no reliable cap.
SendGrid	A paid plan would be needed to restore email; the existing key is lost.
Oxford Dictionaries	Its successor API is a paid commercial service.
Domain name (myvirs.com)	About $10 – $20 per year (the same for any setup)

Hidden costs (people's time):

- Manual deployments: build the frontend, copy it into the backend, package the jar, upload the Elastic Beanstalk bundle.
- Keeping the server's operating system, Java runtime and Elastic Beanstalk platform patched.
- Installing and maintaining Tesseract on the server.

- Safeguarding the Jasypt vault key (if it's lost, production access is lost).
- Separate setup for every developer: Node 8, JDK 8, a local database and Tesseract.

---

10. Why the proposed update is better

The proposed setup keeps the same features and word data but runs as one modern TypeScript application:

- **Hosting:** Vercel.
- **Database:** a managed, serverless Postgres from Neon.
- **Services:** a supported email service replaces SendGrid/Gmail; AWS Textract stays for images.

10.1 Cost

Item	Original setup (estimated)	Proposed setup
Application hosting	$15 – $50 / month (always-on EC2, plus any load balancer)	**$0.** Vercel Hobby plan (free for non-commercial use); Pro is about $20/month per member if needed
Database	$15 – $28 / month (RDS MySQL)	**$0.** Neon free tier; a word database of this size (tens of thousands of rows) fits well within it; paid plans are usage-based if outgrown
Disk, backups, IP address	$5 – $10 / month	$0 (included)
Email	Paid SendGrid plan needed	$0 on a free email tier at this volume
Image OCR (Textract)	About $1.50 per 1,000 images, no working cap	Same per-image price, with a limit that can actually be enforced
Estimated hosting total	**about $35 – $95 / month**	**$0 / month on free tiers**

Free-tier limits and prices change. Check the current Vercel and Neon plans, and whether the project counts as non-commercial, before relying on the free tiers.

10.2 Beyond cost

Area	Original setup	Proposed setup
Deployment	Manual: build, copy, package, upload a bundle	Automatic on every push to GitHub, with a preview link for every change
Maintenance	Patch the server OS, Java runtime and native OCR libraries	No servers to maintain; the platform handles runtime updates, HTTPS and scaling
Reliability	One always-on server; downtime during deploys or failures	Managed, auto-scaling platform; instant rollback to a previous deploy

Area	Original setup	Proposed setup
Security	Readable passwords, open user endpoints, shared admin password, end-of-life libraries (6.3)	Hashed passwords, per-user sessions and roles, supported and patched libraries
Database	Implicit schema, MySQL-only; dev and production differ	Versioned schema migrations; the same Postgres everywhere; instant database copies (branches) for safe testing
Developer setup	Node 8 + JDK 8 + Maven + local database + Tesseract	One language (TypeScript) and one install command, on current Node
Hardware	x86 only (OpenCV has no ARM support)	Runs anywhere, including Apple Silicon
Features	Working, except image upload and dictionary lookups; plus the defects in 6.2	Same features on the same word data, with those defects fixed
Future work	Hard to hire for or update (Angular 4, Spring Boot 1.5)	Current, widely used frameworks with active support